Machine Learning

- A computer program is said to learn from experience E with respect to some class of tasks T and performance measure, if its performance at tasks in T, as measured by P, improves with experience E.
  — Tom Mitchell.

Tasks in ML
- Classification or Pattern Recognition.
- Regression or Prediction.
- Density Estimation.
- Clustering
- Synthesis or Sampling
- Ranking
- Recommendation systems
- Anomaly Detection / Data Mining

Performance (P)

- A quantitative measure to evaluate performance.
  - Usually task specific.

Classification
- Accuracy / Error Rate
- Weighted versions of Accuracy (when data is unbalanced)

Regression
- Mean Squared Error and Other Error measures.
- Application dep: Want some large errors or many medium errors

Density Estimation
- log-prob assigned to examples

Always measured in terms of "generalisation error" (On the test set)

Experience (E)

- Supervised learning
  Labelled data - (data, target value)
  ↓
  Category / class labels, real value, real vector

- Classification
- Regression

Unsupervised learning
  Only data, no labels

  clustering, density estimation

Semi-Supervised learning
- some labelled, most is unlabelled

Multiple-Instance learning

Reinforcement learning
  No examples, reward function
  Payoff for actions

Web search & chatbots
Adobe : OCR Optical Character Recognition
Amazon's Rec system.
ADAS, self drive.

Incomplete history of learning.

Turing Test. 1950.
Rosenblatt's Perceptron 50's
- Math analysis of thinking
  key idea behind SVMs and NN

Construction of fundamental of learning Theory
1960-70.
- Generalisation
  Performance on unseen data.
- Regularisation for ill posed problems.
  - Lin eq for ill-cond. matrices.

Neural Networks (1980s)
- Connectionism
- Back Propagation
- CNN, RNNs.

Learning Theory, SVMs (1990's)
- Margin Maximisation
  Kernel Methods for non-linearity.

Deep Learning >2006
Hinton, Bengio, Le Cun.
DL based solutions

LL / MultiOp / World Models.

Statistical Modelling and Inference.

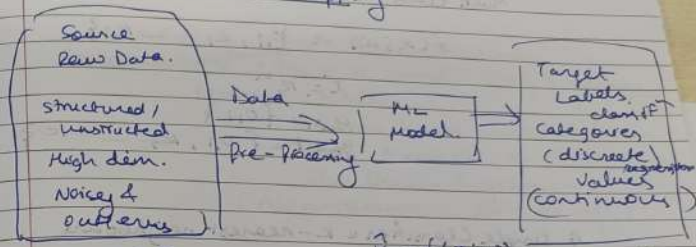
Regression, Hypothesis Testing
Significance Testing

Signal Processing (Data is Signal)
- Detection and Estimation
- Hypothesis Testing
  Classification / Detection
  Estimation
- Regression - Error min

- Representation of Signals
  Fourier, Wavelets etc.
  Compressed sensing, Sparse rep, Dictionary learning.

Optimisation
Min / Max.

ML as a function Mapping



$x \in X$ $\quad \hat{y} = f(x; w)$ $\quad w \in \Omega$ $\quad y \in Y$.

So in supervised learning
Need to predict labels of data.

Regression

- Regressors: function Mapping.
- Regression
  Predict [height / weight / age] based on [picture / video] of person.

2) $f(x; w) : R^n \to R^m$.

$X = R^n \quad Y = R^m$.

Classification

- Also a function mapping.

Binary classification
- gender / sex prediction based on facial images / video

$f : (X, w) \to R^n \to \{-1, 1\}$

Multi class

$f(x; w) \rightarrow \{0, 1, 2, \ldots K\}$

$x = R^n$

$y_{bin} = \{0, 1\}$

$y_{mult} = \{0, 1, 2, \ldots K\}$

A simple classifier: K-nearest neighbours



$K = 3$. → Hyperparameter

→) Majority class △.

KNN for

Can be based on different metrics of 'nearness'.

- Sort all points by distance, choose top K nearest.

KNN has to store all of the data at a time unlike traditional ML models.

KNN for regression

- KNN is a non-parametric model.
- Instance based (lazy) learning

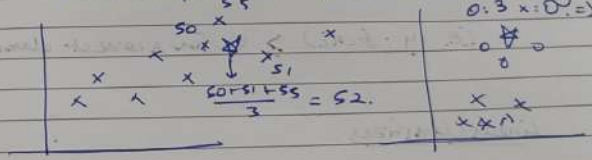
Parametric models:

Assume fixed functional forms with fixed no. of parameters. like linear regression.

(Non-parametric don't do this).

Looks at instances around a new point for prediction

Here computation before prediction - Lazy.



Regression Classification.

KNN problems

- The distance / similarity should be meaningful and capture the notion of the class:

Dist between pixels may not reflect info about face's ID.

KNN hard to scale

All data needs to be stored

Each unseen point needs it's K nearest neighbours to be calculated.

Alternatives to ICAN.

Binary classification.

$(x_1, y_1)\ (x_2, y_2)\ \dots\ (x_N, y_N)$

$x_i \in \mathbb{R}^d$

$y_i \in \{-1, 1\}$

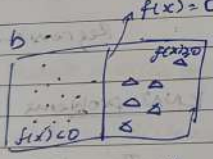
Learn $f(x)$ s.t.

$$f(x_i) = \begin{cases} \geq 0 & y_i = +1 \\ < 0 & y_i = -1 \end{cases}$$

i.e. $y_i f(x_i) \geq 0$ for a correct classification

Linear Classifiers

$f(x) = w^T x + b$



- In 3D discriminant is a plane
- In nD : " " " hyperplane.

where $f(x) = 0$.
(where it's undecided)

In ICNN needed to carry train data.
For a lin classifier, need train data to learn w and then only w is needed for classifying new data. discarded

Learning machine has to mimic the supervisor
(i.e. be able to predict labels / values for data).

3 components

- Raw Input Data.
  Random vector x_i i.i.d. from a fixed but unknown distribution $p(x)$.
- Target Labels:
  A supervisor (oracle) which gives output vector y_i for each input vector x_i, as per conditional distribution $p(y|x)$, fixed but unknown
- Learning machine that can implement a set of functions.
  $\hat{y} = f(x; w) \quad w \in \Omega$
  ↓
  predicted.
- We need to choose the function which best approximates supervisor's response.
- Selection is based on training samples (x_i, y_i) @ $i = 1, \dots N$.

Regression Examples

LiDARs and Point Clouds
Laser based range & distance sen

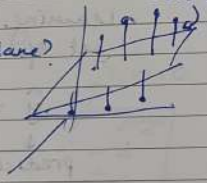
- Columns of lasers on a spinning motor generate a "point cloud"
Problem: find road surface.
Assume planarity
↳ Road is flat and can be ~~as~~ represented by a plane.

Given points estimate params.

dimension ($p=2$) $x_i = (x_{i1} \; \ldots \; x_{ip})^T$
training samples $(x_1, y_1) \; \ldots \; (x_N, y_N)$
parameters $w = (w_0, w_1, \ldots w_p)^T$

$$y = f(x, w) = w_0 + \sum_{j=1}^{p} x_j w_j + \varepsilon$$

ε ↓ error.



How good is fitted plane?

Total Error =

$$\sum_{i=1}^{N} L(y, f(x, w))$$

$$2 \sum_{i=1}^{N} (y_i - f(x_i, w))^2$$

$$= \sum_{i=1} \left(y_i - w_0 - \sum_{j=1}^{p} x_{ij} w_j \right)$$

10 ~~cost~~ : governs function f.
Degree of polynomial + coefficients

Loss Functions

- Choose best function: Minimise a loss (cost) between supervised discrepancy and learning machine response, given input x.

$$L(y, f(x, w))$$

- As want to minimise the loss L over all samples, we ~~want~~ minise expected loss over all possible x and y.
Not just the samples alone.

2) ~~Risk~~ ~~er~~

$$R(w) = \iint p(x, y) \, L(y, f(x, w)) \, dx \, dy$$

$x \in \mathbb{R}$.

↓ Risk (expected loss)

$p(x, y)$: data, target.

$\mathbb{E}[L(y, f(x, w))]$

Defined over all x & y (not only train)

We use empirical risk to estimate $R(w)$ using training samples

Loss functions - Regression

Least squares (L_2 norm) minimisation

y takes continuous vals

$$L(y, f(x,\omega)) = \|y - f(x,\omega)\|_2^2$$

$$\text{if } \|x\|_p = \left(\sum_{i=1}^{d} |x_i|^p\right)^{1/p}$$

L_1 norm

$$L(y, f(x,\omega)) = \|y - f(x,\omega)\|_1$$

Huber loss (robust regression)

Classification Examples

Preprocessing

↓

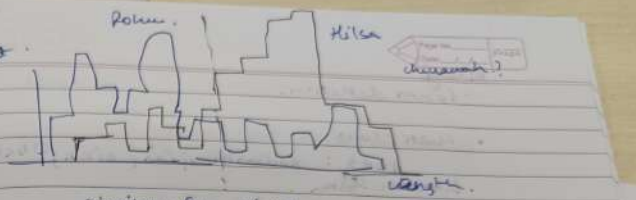
Feature Extraction → length / width / brightness

↓

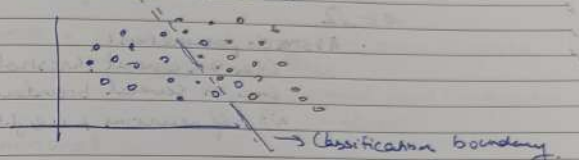
classification

Rohu Hilsa

fish types



similar for lightness.



Model complexity also matters

Loss functions - classifications

Binary classification with equal weights on misclassification.

Min 0-1 loss.

$$L(y, f(x,\omega)) = \begin{cases} 0 & \text{if } f(x,\omega) = y \\ 1 & \text{if } f(x,\omega) \neq y \end{cases}$$

Unequal weights on misclassify.

Min $0-10^7-500$ loss

$$L(y, f(x,\omega)) = \begin{cases} 0 & \text{if } y = f(x,\omega) \\ 10^7 & \text{if } y=1,\ f(x,\omega)=0 \\ 500 & \text{if } y=0,\ f(x,\omega)=1 \end{cases}$$

Binary
- Spam detection.

- Multi class
  ADAS : Pedestrian, Car, Bike, Obstacles
  biometric System.

$w \in \Omega$
- Abstract parameters:
  Ones or several thresholds
  are or several boundaries
  No. of neurons + weights

Risk Minimisation

Find minimiser of the risk functional

$$R(w) = \mathbb{E}_{x,y}[L(y, f(x,w))]$$

$$= \iint_x \int_y L(y, f(x,w)) p(x,y) dx dy.$$

$\Rightarrow f(x,w^*)$ that minimises $R(w)$ over class of function $f(x,w)$, $w \in \Omega$

- $p(x,y)$ is unknown
- Only have the training set
  (x_i, y_i) $i = 1, \ldots, N$.

Function
- $y(x)$
  takes input value/vector x
  outputs y.

$y = mx + c$

$y = x^2$.

- Function minimisation over a space of parameter values
  (finding optimal point/parameter values)

Functional
- $F[p]$
- input any function $p(x)$
  outputs value F.
  Maps function to a real value.

Entropy

$$H[p] = -\int p(x) \ln p(x) dx$$

Risk $R(f) = \mathbb{E}_{x,y}[L(y, f(x))]$ (on ourcore we restrain $f(x) \to f(x;w)$)

- Functional risk minimisation over a space of functions.
  (finding optimal functions)

Empirical Risk Minimisation

$$R(w) = \iint_x \int_y p(x,y) L(y, f(x,w)) dx dy$$

$w \in \Omega$.

Risk functional is replaced by the empirical risk functional.

(Estimate risk functional using training samples)

$$R_{emp}(w) = \frac{1}{N} \sum_{i=1}^{N} L(y_i, f(x_i, w))$$

Learning theory:

1) conditions of convergence
2) How fast is convergence rate?
3) Control generalisation + How
4) construct algorithms to control generalisation
   ↓
   l_2, l_1.

Regression

$$L(y, f(x,w)) = \|y - f(x,w)\|_2^2$$

Class.

$$L(y, f(x,w)) = \begin{cases} 0 & f(x,w) = y \\ 1 & f(x,w) \neq y \end{cases}$$

⇒ Min Empirical Risk over all training samples (instead of all x, y)

$$R_{emp}(w) = \frac{1}{N} \sum_{i=1}^{N} L(y_i, f(x_i, w))$$

- Gradient descent for Loss minimization.

Performance evaluation of learning tasks

- How well does a learned model work?
  - Estimating the True Error Rate
    Performance on the Unseen Data.

Metrics

- Classification Accuracy
- Regression: MSE
- Retrieval: Precision / Recall, f1 score.
- Ranking: mean Average Precision.
- Clustering: Normalized Mutual Info.

Entire population is not available.
Only train data, smaller than desired.

Naive: Use all available data.
⇒ May overfit the train data
Higher chance w/ high-capacity models.
True error rate is underestimated.
Not uncommon to have 100% acc on train data.

Loss functions: Probabilistic views

$$\text{Error} = L(y, f(x,\omega)) = \sum_{i=1}^{N} (y_i - f(x_i,\omega))^2$$

$$= \left(\sum_{i=1}^{N} \left(y_i - \omega_0 - \sum_{j=1}^{d} \omega_j x_i^j\right)^2\right)$$

$$y_i = f(x_i,\omega) = \omega^T x_i$$

$$f(x_i,\omega) = \omega_1 x_i^1 + \omega_2 x_i^2 + \cdots + \omega_d x_i^d$$

Take noise as $N \sim (0, \sigma^2)$.

$$y_i = f(x_i,\omega) + \epsilon_i$$

$$= \omega^T x_i + \epsilon_i$$

$$\epsilon_i = y_i - \omega^T x_i$$

1) N iid errors $\epsilon_1, \epsilon_2, \cdots \epsilon_N$

2) $P(\epsilon_i) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2}\frac{(\epsilon_i - 0)^2}{\sigma^2}}$

$$= \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2}\frac{(y_i - \omega^T x_i)^2}{\sigma^2}}$$

2) The joint probability of observing all N iid errors.

1) $P(\epsilon) = P(\epsilon_1, \epsilon_2, \cdots \epsilon_N)$

$$= \prod_{i=1}^{N} P(\epsilon_i)$$

$$= \prod_{i=1}^{N} \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2}\frac{(y_i - \omega^T x_i)^2}{\sigma^2}}$$

$$= \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2\sigma^2}\sum_{i=1}^{N}(y_i - \omega^T x_i)^2}$$

$$= \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2}\frac{\|y - X^T\omega\|_2^2}{\sigma^2}}$$

$X = [x_1\ x_2 \cdots x_N]_{d \times N}$

$y = [y_1 \cdots y_N]_{N \times 1}$

3) $P(\epsilon_1, \epsilon_2, \cdots \epsilon_N)$ as a function of parameters.

$$L(\omega) = P(\epsilon_1, \epsilon_2, \cdots \epsilon_N | \omega)$$

$$= \frac{1}{(\sqrt{2\pi\sigma^2})^N} e^{-\frac{1}{2}\left(\frac{\|y - X^T\omega\|_2}{\sigma}\right)^2}$$

4) Likelihood.

5) Gaussian prior leads to an l_2-norm.

2) Assuming Laplacian Prior.

* Error $= L(y_i, f(x_i, w))$

$$= \sum_{i=1}^{N} |y_i - f(x_i, w)|$$

- Noise is zero mean Laplacian with scale b.

$$\Rightarrow p(\epsilon_i) = \frac{1}{2b} e^{-\frac{|\epsilon_i|}{b}}$$

$$= \frac{1}{2b} \exp\left(-\frac{|y_i - w^T x_i|}{b}\right)$$

∴ N iid errors. $\epsilon_1, \ldots, \epsilon_N$

⇒ Probability of observing them together

$$P(\epsilon_1, \epsilon_2, \ldots, \epsilon_N | w) = \prod_{i=1}^{N} \frac{1}{2b} e^{-\frac{|y_i - w^T x_i|}{b}}$$

$$= \frac{1}{(2b)^N} e^{-\frac{\sum (y_i - w^T x_i)}{b}}$$

2) $f(w) = \frac{1}{(2b)^N} e^{-\frac{\|y - X^T w\|_1}{b}}$

↓ Likelihood function. ↳ View this as a function of parameters.

MLE

- Maximise likelihood over all available samples

$$\hat{w} = \arg\max_w f(w) = \arg\max_w \frac{1}{(\sqrt{2\pi}\sigma)^N} \exp\left(-\frac{\|y - X^T w\|_2^2}{\sqrt{2}\sigma}\right)$$

Max log-likelihood as log is a monotonic function.

$$\hat{w} = \arg\max_w \underbrace{\log p(\epsilon_1, \epsilon_2, \ldots \epsilon_N | w)}_{\text{data}} \quad \underbrace{\text{Param.}}$$

$$= \arg\max_w \frac{-1}{2\sigma^2} \|y - X^T w\|_2^2 + \text{const.}$$

Valid for an arbitrary distribution

$$\hat{w} = \arg\max_w \log P(\epsilon_1, \ldots \epsilon_N | w)$$

$$= \arg\max_w \sum_{i=1}^{N} \log p(\epsilon_i | w)$$

Logistic Regression

- Regression used for a binary (discrete) target variable $y \in \{0, 1\}$.
- Linear Regression: for a continuous, normally distributed target $y \in \mathbb{R}^m$.

Function to map continuous values to 0,1

- Thresholding using some squishy operation

So now instead of gaussian y_i : ~~Binomial~~ Bernoulli y_i (generally as a Binomial RV) (vector of 0, 1s)

Bernoulli: One trial for a binary experiment.

$$P[y \mid x; p] = \begin{cases} p & y = 1 \\ 1-p & y = 0 \end{cases}$$

$$= p^{y}(1-p)^{1-y}.$$

Binomial - n trials of Binary experiments

Bernoulli: $n = 1$

$$\Rightarrow P(4 = y \mid x; n, p) = {}^{n}C_{y}\, p^{y}(1-p)^{n-y}$$

When x_i are discrete / mixed / grouped

1 dimension ← ungrouped: Bernoulli

~~Multiple dim~~ ← grouped: Binomial

↳ Binned, Categorical, discrete

x = age group ——→ [18; 20]

y = # people voted

Many individuals can have same x

∴ Binomial.

Why not use Linear Regression for Binary Responses

- Response 4 is not normally distributed.

Variability of 4 is not constant.

- $\mathrm{Var}(4)$ depends on $E[4]$.

Linear assumes normal distribution with a fixed variance.

For a $4 \sim \mathrm{Binomial}(n, p)$

$\mathrm{Var}(4) = np(1-p)$

which depends on $E[4] = np$.

2) Spread of 4 is not constant.

- Model has to produce fitted probabilities between 0 and 1

Lin Regression predicts fitted responses from $-\infty$ to ∞.

Linear vs logistic

Linear regression tries to predict mean of 4. i.e. $f(x, \omega) = \omega^{T}x$.

$$E[4] = E[f(x, \omega)] + \epsilon.$$

$$= E[f(x, \omega)] + 0.$$

$$= f(x, \omega). \rightarrow \text{deterministic function.}$$

Logistic Regression: Predicts p.

Probability of a trial resulting in success. $(4 = 1)$

$p \in [0, 1]$ is better than $[0, 1]$.

More informative of confidence.

What function of p to model.

$$\text{Odds}(Y=1) = \frac{P(Y=1)}{P(Y=0)} = \frac{P(Y=1)}{1-P(Y=1)}$$

$$\text{Odds}(Y=1) = \frac{p}{1-p}$$

Hence can go from odds to p and p to odds

$$\Rightarrow p = \frac{\text{odds}(Y=1)}{1+\text{odds}(Y=1)}$$

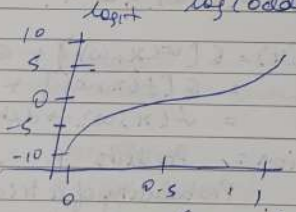
logit function $[\log(\text{odds})]$

Why?

Odds maps $p \in [0,1] \rightarrow [0, \infty)$

$\log \text{odds} : [0, \infty) \rightarrow [-\infty, \infty]$

2) Now can use linear regression to predict $\log(\text{odds})$.



$\Rightarrow$ Logistic Regression = Linear Regression on $\log(\text{odds})$

$$\log(\text{odds}) = w_0 + w_1 x_1 + \cdots + w_d x_d = w^T x.$$

$$\text{logit}(p) = \log\left(\frac{p}{1-p}\right) = w^T x$$

$$\frac{p}{1-p} = e^{w^T x} = e^z$$

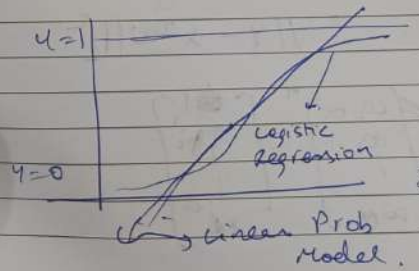
$\Rightarrow$ Recovering $p = P(Y=1)$.

$$p = \frac{e^{w^T x}}{1+e^{w^T x}} = \frac{e^z}{1+e^z} = \left(\frac{1}{1+e^{-z}}\right)$$

sigmoid function $\sigma(z) : R \Rightarrow [0,1]$

$$1-p = \frac{1}{1+e^{w^T x}} = \frac{1}{1+e^z} = \frac{e^{-z}}{1+e^{-z}}$$

Logistic Regression vs Linear Regression.



$$P(Y=1 \mid X=x) = \frac{1}{1+e^{-z}}$$

$$z = w^T x.$$

MLE

$$P(y \mid x; p) = \begin{cases} p & y = 1 \\ 1-p & y = 0 \end{cases}$$

$$l(w) = \sum_{i=1}^{N} y_i \log p(x_i; w) + (1-y_i) \log (1 - p(x_i, w))$$

$$= \sum_{i=1}^{N} y_i \log \frac{p(x_i, w)}{(1-p(x_i, w))} + \log(1 - p(x_i, w))$$

$$= \sum_{i=1}^{N} y_i \log \left(\frac{e^{w^T x_i}}{1+e^{w^T x_i}} \Big/ \frac{1}{1+e^{w^T x_i}} \right) + \log \left(\frac{1}{1+e^{w^T x_i}} \right)$$

$$= \sum_{i=1}^{N} y_i w^T x_i - \log\left(1 + e^{w^T x_i}\right)$$

MLE for Linear Regression

$$L[y, f(x, w)] = \sum_{i=1}^{N} (y_i - f(x_i, w))^2$$

$$= \| y - x^T w \|_2^2$$

$$y_i = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_d \end{bmatrix}^T \begin{bmatrix} 1 \\ x_i \\ \vdots \\ x_d \end{bmatrix}$$

$w \in R^{d \times 1}$, $x \in R^{d \times N}$, $x^T w$, $x_i \times R^{d \times 1}$, $w^T x_i$ or $x^T w_i$

$$y_i = w_0 + w_1 x_i^1 + w_2 x_i^2 + \dots + w_d x_i^d$$

$$\frac{\partial}{\partial w_j} l(w) = \sum_{i=1}^{N} y_i x_i^j - \frac{e^{w^T x_i}}{1+e^{w^T x_i}} \times x_i^j$$

$$= 0$$

$$\Rightarrow \sum_{i=1}^{N} x_i^j \left(y_i - \frac{e^{w^T x_i}}{1+e^{w^T x_i}} \right) = 0$$

$$\Rightarrow \log\left(\frac{p}{1-p}\right) = w^T x_i$$

$$p = \frac{e^{w^T x_i}}{1+e^{w^T x_i}}$$

$$\ln\left(\frac{p}{1-p}\right) = w^T x_i$$

$$\frac{p}{1-p} = e^{w^T x_i}$$

$$p = e^{w^T x_i} - e^{w^T x_i} p$$

$$= \frac{1}{1+e^{-w^T x_i}} \sim \sigma(w^T x_i)$$

$$\Rightarrow \sum_{i=1}^{N} x_i^j \left(y_i - \frac{1}{1+e^{-w^T x_i}} \right)$$

$$\Rightarrow \sum_{i=1}^{N} x_i^j \left(y_i - p(x_i; w) \right) = 0$$

This is not a closed form normal equation (unlike linear regression)
$(X^T X)^{-1} X^T y$

Gradient Ascent

$$w_j^{t+1} = w_j^t + \eta \frac{\partial l(w)}{\partial w_j}$$

$$= w_j^t + \eta \sum_{i=1}^{N} x_i (y_i - p(x_i; w))$$

Iteratively updating weights in this way increases likelihood each round

∴ Eventually reach the maximum where the change in weights are small

Hence we can stop when the sum of absolute values of differences is less than some small number.

This method helps us to model binary outcomes.

$$p(x_i; w) = \frac{1}{1 + e^{-w^T x_i}}$$

$$p(x_i; w) = \frac{1}{1 + e^{-w^T x_i}}$$

with no x

$\Rightarrow P(y=1) = p$ → constant

Evidence changes this

$$\|x\|_p = \left(\sum_{i=1}^{d} |x_i|^p \right)^{1/p} \quad (L_p\text{-norm})$$

Performance Evaluation of Learning Tasks

- How well a learned model works is estimated by the TRUE ERROR RATE. Classifier's error rate on entire population.

Metrics

Classification: Accuracy
Regression: MSE
Retrieval: Precision | Recall, F1 Score
Ranking: Mean Avg Precision
Clustering: NMI

Regression Metrics

Mean Squared Error

$$L(y, f(x,w)) = \frac{1}{N} \sum_{i=1}^{N} (y_i - f(x_i, w))^2$$

$$= \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y})^2$$

Mean Absolute Error

$$L(y, f(x,w)) = \frac{1}{N} \sum_{i=1}^{N} |y_i - y_i|$$

Root mean squared error,
Max Error (L_∞-norm)

L_∞ i.e. $\|x\|_\infty = \max_{1 \le i \le n} |x_i|$

Non neg $\|x\|_\infty \ge 0$ $\|x\|_\infty = 0$ iff $x = 0$

Limit of Lp norms

$$\lim_{p \to \infty} \left(\sum_{i=1}^{d} |x_i|^p \right)^{1/p} = \max_{1 \le i \le d} |x_i|$$

Classification

True Positive (TP): Predicts positive and y is positive)

False Positive (FP)
- False Alarms

- True Negative (TN)

- False Negative (FN)
  - 'Misses'

Confusion Matrix

Pred / Actual	Positive	Negative
Positive	TP	FN
Negative	FP	TN.

- Metrics are usually function of TP, FP, FN, TN

Depending on Application
Method of computing TP, TN, FP & FN may change

for classification

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN}$$

$$= \frac{TP + TN}{(\#\ \text{Samples})}$$

$$\text{Precision (P)} = \frac{TP}{TP + FP}$$

$$\text{Recall (R)} = \frac{(TP)}{(TP + FN)}$$

$$F1\ \text{score} = 2 \times \frac{(\text{Precision} \times \text{Recall})}{(\text{Precision} + \text{Recall})}$$

Classification Metrics - Imbalanced Data.

Not all evaluation metrics are same.
Choice often is dependant on application & the quality of data.
↳ May be imbalanced
Such as cancer diagnoses are relatively scarce.

Class 0 : Healthy
9900

class 1 : Individuals with Cancer
100.

Trivial: All people are Healthy (Class 0)

Actual \ Pred	Class 0	Class 1
Class 0	9900 TP	0
Class 1	100 FP	0

Actual \ Pred	Class 1	Class 0
Class 1	0 TP	100 FN
Class 0	0 FP	9900 TN

$$\text{Accuracy} = \frac{0+9900}{100+9900} \times 100$$

$$= \frac{99}{100} \times 100$$

$$= 99\%$$

$$\text{Precision (P)} = \frac{TP}{TP+FP}$$

$$= \frac{0}{(0+0)}$$ undefined NaN

$$\text{Recall} = \frac{TP}{TP+FN} = 0\ \%$$

$$\text{F1-Score} = \frac{2PR}{P+R} = \frac{0}{\text{NaN}}$$

= Not defined.

Bad Classifier: Most samples in class 0

Actual \ Pred	Class 1	Class 0
Class 1	1 TP	99 FN
Class 0	1 FP	9899 TN

$$\text{Acc} = \frac{1+9800}{10000} = 99\%$$

$$\text{Precision} = \frac{TP}{TP+FP} = \frac{1}{1+1} = \frac{1}{2} = 50\%$$

$$\text{Recall} = \frac{TP}{TP+FN} = \frac{1}{100} = 1\%$$

$$\text{F1 Score} = \frac{2 \times (P \times R)}{P+R} = 2 \times \frac{0.5 \times 0.01}{0.51}$$

$$= \frac{0.01}{0.51}$$

$$\approx 2\%: 0.02$$

Perfect Classifier

Actual \ Pred	Class 1	Class 0
Class 1	TP 100	0 FN
Class 0	FP 0	9900 TN

$\Rightarrow$ Acc = 100%.

P = 100%.

R = 100%.

F1 = $\frac{2 \times 1 \times 1}{1+1}$ = 1

(ROC)

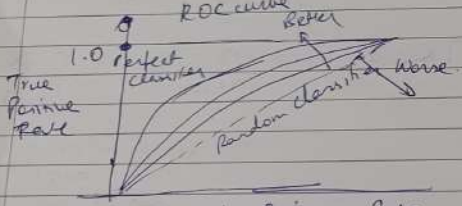
Receiver Operating Characteristic curve.

- From the early days when detecting objects using a RADAR receiver.

True Positive Rate => Recall = $\frac{TP}{TP+FN}$
(TPR) (Sensitivity)

False Positive Rate (FPR) = $\frac{FP}{FP+TN}$
(Predicted)

=> The samples that should be as class 0. How bad is the classifier at predicting class 0.



FPR = 1 − Specificity

$1 - \left(\frac{TN}{FP+TN}\right)$ How good it predicts class 0

How to Plot ROC curve for a binary classifier.

- Say classifier generates $P(y=1|x)$
- Set a threshold $\tau \in [0,1]$ on $P(y=1|x)$ over its range ex $\{0, 0.01, 0.02, \ldots, 1\}$.
- Compute TPR & FPR for each τ.
- Plot the ROC curve
- Compute area under curve (AUC-ROC)

$\hat{y}_i = \begin{cases} 1, & \text{if } p_i \geq \tau \\ 0, & \text{o.w.} \end{cases}$

=> Random Binary classifier has AUC-ROC = 0.5

similarly other plots - Precision vs Recall etc.

ROC curve helps us to pick the threshold

Control how many FP we are allowed to have

A perfect one has AUC = (ROC) = 1
i.e. Recall / TPR = 1
FPR = 0.

So for each $\tau \in \{0, 0.01, \ldots, 1\}$
we calc
$(FPR(\tau), TPR(\tau))$
=> we plot this

=) Plot the path of points as τ goes from 1 to 0

=) $\tau = 1$ model classifies everything as class 0

=) $TP = 0$ $FP = 0$

=) $TPR = 0$

$TPR = 0$.

$\tau = 0$ everything as class 1.

=) $FN = 0$

=) Recall = 1

$TPR = \frac{TP}{TP+FP}$ =) $FPR = \frac{FP}{FP+TN}$

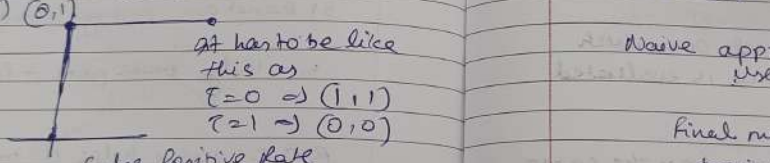
Actual \ Predicted	class 1	class 0
class 1	TP	FN
class 0	FP	TN

$\tau = 0$ =) All class 1 $\tau = 1$ =) All class 0. =) (1,1)

=) FN = 0 TN = 0 (0,0) (0,1)

=) Perfect Recall = $\frac{TP}{TP+FN} = 1$ True Positive Rate.

Precision = $\frac{TP}{TP+FP}$



It has to be like this as

$\tau = 0$ =) (1,1)

$\tau = 1$ =) (0,0)

The ROC curve is monotonic (increasing / non-decreasing) because as we go from $\tau = 1$ to 0, we classify more positives =) FP, TP can only increase.

$$\frac{d\,TPR}{d\,FPR} = \frac{P(\text{score} \mid y=1)}{P(\text{score} \mid y=0)}$$

A good model initially (wrt τ, let's say 0.7 to 1.0) gives more weightage to the class 1 than 0 (to predict 1 well as +ve means 1) =) The model's TPR goes up without FP going up a lot

=) Steep ascent.

As we pass the point where we have most of the TPs, we get more FP =) ROC becomes flatter.

- Entire population is not available for evaluating performance.

Finite set of train data.

Naive approach

Use all available data.

Final model tends to overfit to the training data.

More chances with high capacity models (Neural Networks)

True error is underestimated

- Can get 100% acc on training data.

~~Data T~~

Data Terminology

Training Data (Train set)

- Over which we 'train' model
  ('Train' = 'Maximise the likelihood')
  'Minimise the 'loss'
  to find the best model parameters.

Val set

Over which "hyperparams" are tuned, & comparisons are made,
But models are not trained.

Test Data

- "Unseen" data set over which model performance is evaluated.

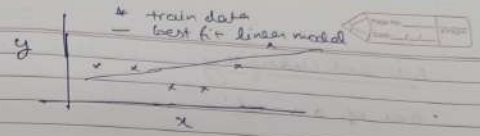
All data sets are drawn from the same data distribution.

Model fitting

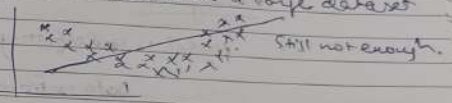
$y = w_0 + w_1 x$. First Order Curve

Observations

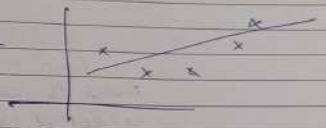
- High err on Train set
  High error on val set



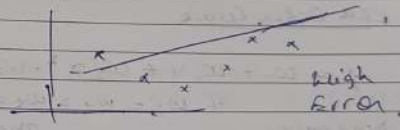
Fitting linear models on a large dataset



Removing noise
Still nothing



2) ~~Train~~ Test Set



2) Sources of Error?

- Small no. of train samples? Nope
- Noise in train data Nope.
- Insufficient Model Capacity?
  Too few parameters to capture data complexity.

2) Line used to model points along a quadratic.

3) Model 'underfits', not capable of sufficiently expressing the training data.

Bias and Underfitting

- Bias of a model:
  - Even with a very large training set val/test error is high.
  - ⇒ Model is underfitting.

Data is from a more complex distribution and model is too simple, hence high error (train, val)

Noise is reduced to 0 as if we have a lot of samples (if zero mean)

Fifth Order Curve

$y = w_0 + w_1 x + w_2 x^2 + w_3 x^3 + w_4 x^4 + w_5 x^5$

If $w_3 = w_4 = w_5 = 0$

Then can fit quadratic

Digression

Linear Model:

Yes, but in w. (not in x)

Question should be is $f(x, w)$ linear in w.

We learn w, x is constant - input.

⇒ This method of taking powers of data and combining it to a linear model - polynomialising.

1 dim → 6 dim space

But still linear

We could have zero train error



Linearity

Function $g(u)$ is linear if:-

a) $g(u+v) = g(u) + g(v)$ Superposition

b) $g(cu) = c\,g(u)$ Homogeneity.

Our linear model of $y = f(x, w) = w^T x$ is linear in x and in w.

for x

a) $f(x_1 + x_2, w) = w^T(x_1 + x_2)$

$= w^T x_1 + w^T x_2$

$= f(x_1, w) + f(x_2, w)$

b) $f(cx, w) = w^T(cx)$

$= c\,w^T x = c\,f(x, w)$

for w

a) $f(x, w_1 + w_2) = (w_1 + w_2)^T x$

$= w_1^T x + w_2^T x$

$= f(x, w_1) + f(x, w_2)$

b) $f(x, cw) = (cw)^T x$

$= c\,f(x, w)$.

$y = f_1(x, w_0, w_1) = w_0 + w_1 x$

= Not linear

$f_1(x_1 + x_2, w_0, w_1) \neq f(x_1, w_0, w_1) + f(x_2, w_0, w_1)$

As $f(x, w_0, w_1)$ linear in w. ✓

i's $y = w_0 + w_1 x + \dots + w_5 x^5$

linear in x? No

linear in w? Yes.

So we got a curve with perfect fit to train

=> zero train error

low-ish test/val error

- This is lucky.

But very diff from ground truth quadratic curve.

The model 'overfits' train data.

It fits to the noise in that specific train set.

Hence,

if we train/fit 5 degree models on different datasets of the train data, we get very different models.

=> High variance - Overfitting.

fits to Nois

Low train, high val, test error

$\left(\sum_{i=1}^{n} |x_i|^p\right)^{1/p}$

- Typical for high capacity (many parameters) models.

Several mitigation approaches to reduce variance and hence overfitting.

1) Use a larger train set

might be expensive computationally / financially or infeasible.

2) Regularization: l_1 - LASSO

l_2 - Ridge.

l_0: No. of non zero elements

$w = w_0\ w_1\ w_2\ w_3\ w_4\ w_5$

Say $l_0 \leq 3$

=> we can fit a quadratic curve.

3) Feature Engineering

Previously models were linear even in x. (in the old era)

Bias Variance Trade Off

1) High Bias / underfitting: Model complexity too low

Simple Model

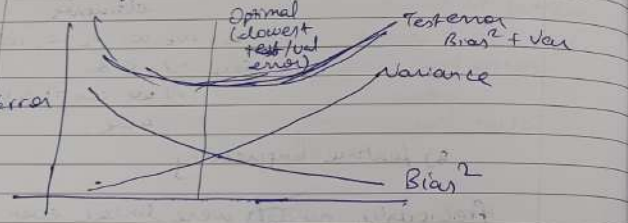
Low variance, High Bias

Train, Test & Val Both too high

2) High Variance / Overfit
Model Complexity is too high.
Train error is low
Test/val error high.
Low Bias.

3) Correct fit. Model Complexity just right.
Both Train & Test/Val err are low enough.

In our case, achieved by fitting quadratic.



Regularisation

$y = w_0 + w_1 x + w_2 x^2 + w_3 x^3 + w_4 x^4 + w_5 x^5$

If $w_3 = w_4 = w_5 = 0$ ✓ Then we're done then?

$\vec{w} = [w_0\ w_1\ w_2\ w_3\ w_4\ w_5]^T$

$\vec{x} = [1\ x\ x^2\ x^3\ x^4\ x^5]^T$

Then learned model's parameters.

Constrained Optimisation

$\vec{w^*} = \arg\min_{\vec{w}} \sum_{i=1}^{N} (y_i - \vec{w}^T \vec{x_i})^2$

s.t. $\|\vec{w}\|_0 \leq 3$

constraint on model params

can solve this to get the quadratic best fit.

Alternatively

$w^* = \arg\min_{w} \sum_{i=1}^{N} (y_i - w^T x_i)^2 + \lambda \|\vec{w}\|_0$

Lagrange Multiplier

Penalises large no. of non zero params.

Regularisation Term: Not differentiable

Relaxed Optimisation problem is used.

$\vec{w^*} = \arg\min_{w} \sum_{i=1}^{N} (y_i - w^T x_i)^2 + \lambda \|w\|_1$

Least Absolute Shrinkage and Selection Operator.

Promotes sparsity

Differentiable (using subgradients) & convex.

Regression Model (Linear)

$y = f(x, w) + \epsilon$ → true. → random noise

$\vec{y} = w^T x$

$\nabla_w L(w) = \frac{1}{N} \sum_{i=1}^{N} (y_i - w^T x_i) x_i$

scalar — input data vector

⇒ Gradient is a linear combination of input data vectors.

Crucial in kernel methods (non linearity)

Weights are linear combinations of the input vectors.

Gradient Descent

$w_{t+1} = w_t - \lambda \nabla_w L(w)$

$\nabla_w L(w)$ → Points in next increase of $L(w)$

w_t → gradient descent.

Minimising $L(w)$

$$\vec{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix}_{N \times 1} \quad X = [x_1, x_2 \dots x_N]_{d \times N} \quad w = [w_0\ w_1 \dots w_d]^T$$

$L(w) = \sum_{i=1}^{N} (y_i - w^T x_i)^2$

$= \| y - X^T w \|_2^2$

$= (y - X^T w)^T (y - X^T w)$

$= y^T y - y^T X^T w - (X^T w)^T y + (X^T w)^T X^T w$

$= y^T y - 2 (Xy)^T w + w^T X X^T w$

⇒ $\nabla_w L(w) = 0 - 2Xy + 2XX^T w = 0$

(For a pos semi-def mat A, $\frac{\partial X^T A X}{\partial x} = 2AX$.)

$\nabla_w L(w)$

⇒ $Xy = (XX^T) w$

$\boxed{(XX^T)^{-1} X y = \hat{w}}$

(Assuming XX^T is invertible) $d \times N \; N \times d$ if $d < N$ then it won't be.

$\hat{w} = (XX^T)^{-1} X y$

$d \times 1 \quad d \times N \; N \times d \quad d \times N \; N \times 1$

(If $X \in \mathbb{R}^{N \times d}$ $(X^T X)^{-1} X^T y$)

$$L(w) = \sum_{i=1}^{N} (y_i - w^T x_i)^2 + \lambda w^T w$$

Prior on w is
$w \sim N(0, \lambda I)$
$\Rightarrow$ we get the $\hat{w}$ by
MAP

$$L(w) = \|y - X^T w\| + \lambda w^T w$$

$$= (y - X^T w)^T (y - X^T w) + \lambda w^T w$$

$$= y^T y - y^T X^T w - (X^T w)^T y + (X^T w)^T X^T w + \lambda w^T w$$

$$= y^T y - 2(Xy)^T w + w^T X X^T w + \lambda w^T w$$

$$= -2Xy + 2XX^T w + 2\lambda w$$

$$= 0$$

$$w(XX^T + \lambda I) = Xy$$

$$w = (XX^T + \lambda I)^{-1}(Xy)$$

$\log(L_{ridge}(w)) =$ -

Machine Learning.

Classification

. Probabilistic Classification (Bayes)

Non parametric Classification (KNN's or IC-nearest-neighbours)

Decision trees.

Linear Regression

$y \sim N[f(x_i, w), \sigma^2)$

$L(y, f(x,w)) = -\sum_{i=1}^{N} \{(y_i, f(x_i, w))\} + R(w)$

$= -l(w) + R(w)$

↳ likelihood.

Regularisation

1) L_2 reg. $R(w) = ||w||_2^2$

Ridge Regularisation.

Gaussian Prior.

- Hoag Helps reduce variance

2) L_1 Regulariser ; LASSO

Least absolute shrinkage and selection operator.

$R(w) = ||w||_1$

- Laplacian Prior.
  - Promotes sparsity
  - Bad when features are correlated.

3) Gaussian + Laplacian Mixture Prior or Elastic-Net

$R(w) = ||w||_2^2 + \lambda_1 ||w||_1$

Logistic Regression

Linear Map + Logit + Bernoulli model.

Bernoulli model

$$P(y|x;p) = \begin{cases} (1-p) & y=0 \\ p & y=1 \end{cases}$$

$$= p^y (1-p)^{1-y}$$

Log-likelihood. $p(x_i; w) = \frac{1}{1+e^{-w^T x_i}}$

$$l(w) = \sum_{i=1}^{N} y \log p + (1-y) \log(1-p)$$

Probabilistic Classifier

Classifier

$\hat{P}(\hat{y}_i = 1 \mid x_i; w) > \hat{P}(\hat{y}_i = 0 \mid x_i; w)$

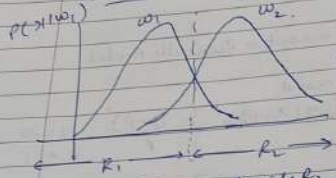
$\hat{y}_i = 1$

$\hat{y}_i = 0.$

for M class classification.

$\hat{y}_i = \underset{w = \{0,1,\ldots,M\}}{\arg\max} P(f(x_i, w) = y \mid x_i, w)$

Decision Boundary



$P(\omega_1|x) = P(\omega_2|x)$

$P(x|\omega_1)P(\omega_1) = P(x|\omega_2)P(\omega_2)$

$\Rightarrow$ If $P(\omega_1) = P(\omega_2)$

$P(x|\omega_1) = P(x|\omega_2)$

$\Rightarrow$ Based on likelihood.

w is perpendicular to the decision boundary.

Discriminative vs Generative Probabilistic Classifier

For Binary classification
we have to predict $y \in \{0, 1\}$

Consequently, in $p(y|x)$

Discriminative classifiers directly model $p(y|x)$
Logistic Regression.

$\Rightarrow$ In discriminative just need class 1 samples to have a high prob of being 1.
and class 0 being low.

$$p(y=1|x;w) = \frac{1}{1+e^{-w^T x}}.$$

- So just discriminate between the two classes

Generative classifier leverages data distribution, given a class

likelihood.

$$P(y|x) = \frac{p(x|y)\,p(y)}{p(x)}$$ ← Regardless of data & the balance of the classes.

Posterior

↳ Not depend on y.
Marginalised over all y.
$\Rightarrow$ Can ignore evidence.
Also it is fixed as we are given the input data.

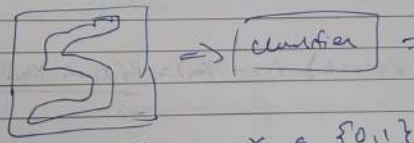
So we can estimate parameters of the distributions and then generate more samples from them. ($p(x|y)$)

- Gen. AI does the same thing.

In discriminative we can't really generate x.

Example Generative Classifier.

Digit Recognition



$\Rightarrow 5.$

$x_1 - - x_n \in \{0,1\}$. $\Rightarrow x_i \in R^{30\times30}$

$y_i \in \{5, 6\}$

The Bayes Classifier

Now a good strategy

$$\arg\max_y P(y \mid x_1, \ldots x_n)$$

How to compute it.

This is kind of obvious

$x_i \in R^{30 \times 30}$

How do we

$$P(y \mid x_1, \ldots x_n) = \frac{P(x_1, \ldots x_n \mid y)\, P(y)}{P(x_1, \ldots x_n)}$$

likelihood, prior, normalisation constant.

Likelihood can be explained / answered.

For each class $P(x_1 \ldots x_n \mid y)$ is how the data was generated.

$$P(y=5 \mid x_1 \ldots x_n) = \frac{P(x_1 \ldots x_n \mid y=5) P(y=5)}{P(x_1 \ldots x_n \mid y=5) P(y=5) + P(x_1 \ldots x_n \mid y=6) P(y=6)}$$

$$P(y=6 \mid x_1 \ldots x_n) \propto P(x_1 \ldots x_n \mid y=6) P(y=6)$$

So we need to learn likelihood and prior

n = no. of pixels in an image.

No of params in $P(x_1, \ldots x_n \mid y)$

$2(2^n - 1)$

x_1	x_2
x_3	x_4

x_1	x_2	x_3	x_4	$P(x_{i=1} \mid y=55)$
0	0	0	0	p_1
0	0	0	1	p_2
0	0	1	0	
				p_{2^n}

2) $(2^n - 1)$ for 1 class

$\{y_1, y_2\}$ we can get $2^{n^{th}}$ value

2) for all k classes $\rightarrow k(2^n - 1) = 2(2^n - 1)$ as $1 - (p_1 + p_2 + \ldots p_{2^n - 1})$

could be a better way to represent this.

But this may still be very hard to estimate.

- computationally expensive

Naive Bayes Model

Assume all features are independent when given the class label y.

$$P(x_1, \dots, x_n | y) = \prod_{i=1}^{n} P(x_i | y)$$

of parameters for modeling

$P(x_1 | y), \dots, P(x_n | y)$

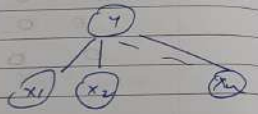
$2n$.

↳ for 1 class

Naive Bayes classifier

Given

- Prior $P(y)$
-



Estimate using frequentist.

$$P(x_i = u | y = v) = \frac{\text{count}(x_i = u \wedge y = v)}{\text{count}(y = v)}$$

Gaussian Discriminant Analysis (GDA)

If input follows Gaussian Distribution

$P(x | y = 0) = \mathcal{N}(\mu_0, \Sigma)$

$P(x | y = 1) = \mathcal{N}(\mu_1, \Sigma)$

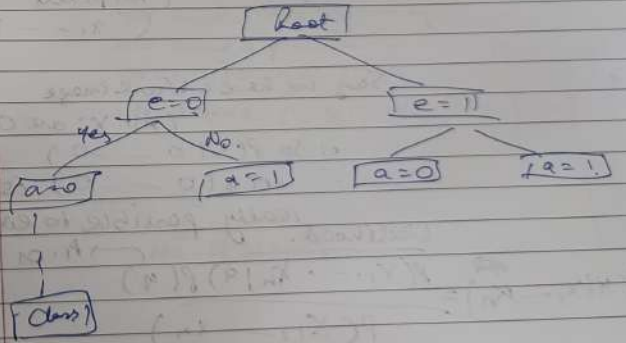
$y \sim$ Bernoulli (ϕ)

⇒ We can compute MLE estimates.

KNN

- Non parametric
- Need to carry every train sample
- Notion of distance
- Voting

Decision Trees



Revision

Discriminative vs Generative Probabilistic Classifier

- Discriminative:
  Directly models $P(y|x)$
  Logistic regression:

$$P(y=1|x;\omega) = \frac{1}{1+e^{-\omega^T x}}$$

Generative:
Models $p(y|x)$ via the input data distribution $p(x|y)$

$$P(y|x) = \frac{p(x|y)\,P(y)}{P(x)}$$

The Bayes classifier

$$\underset{y \in \{5,6\}}{\text{argmax}} \; P(y \mid x_1, \ldots x_n)$$

(All pixels $x_1 \ldots x_{900}$)

Say we have a blank image (All x_i are 0)

c) So $P(5|0 \ldots 0)$
$P(6|0 \ldots 0)$ is not really possible to evaluate.

Likelihood. → Prior

$$P(y|x_1 \ldots x_n) = \frac{P(x_1 \ldots x_n|y)\,P(y)}{P(x_1 \ldots x_n)}$$

- So we can now take a multiple images of class 5 and then use it to estimate/calculate the likelihood?

$$P(y=5|x_1 \ldots x_n) = \frac{P(x_1 \ldots x_n|y=5)}{P(y=5)}$$

$$P(x_1 \ldots x_n|y=5)P(y=5) + P(x_1 \ldots x_n|y=6)$$
$$P(y=6)$$

⇒ No. of params:

$(2^n - 1)$

$x_i \in \{0,1\}$

$x_1 \ldots x_n$

$2 \times \ldots \times 2 = 2^n$ possibilities

Parameters. $(2^n - 1)\left(1 - \sum_{i=1}^{k} P_i\right)$

↓ for one class. ↑ for last

for each combination of $x_1 \ldots x_n$, we need a probability value (parameter) to define $P(x_1 \ldots x_n|y=y_i) = P_i$, $P(x_1 \ldots x_n|y=k) = 1 - \sum_{i=1}^{k-1} P_i$

⇒ $k(2^n - 1)$

↓ No. of classes.

Naive Bayes

Assumption

- Assuming that all n features are independent given the class label y.

$$P(x_1, \dots, x_n \mid y) = \prod_{i=1}^{n} P(x_i \mid y)$$

Now need only n parameters (probabilities) for one class

⇒ $k \cdot n$ parameters (k = # of classes) (2n here)

$y \in \{5, 6\}$

Could be some are dependent, some are independent.

Naïve Bayes

Estimate $P(y = v) = \frac{\text{count}(y = v)}{\#\text{records}}$

Estimate $P(x_i = u \mid y = v)$ as fraction of samples with $x_i = u$ and $y = v$ given $y = v$.

$$P(x_i = u \mid y = v) = \frac{\text{count}(x_i = u \wedge y = v)}{\text{count}(y = v)}$$

GDA

Input follows

- Gaussian Distribution

$P(x \mid y = 0) \sim N(\mu_0, \Sigma)$

$P(x \mid y = 1) \sim N(\mu_1, \Sigma)$

$y \sim \text{Bernoulli}(\phi)$. Find out by MLE.

$P(y_1 \mid x) \geq P(y_2 \mid x)$ classify y_1.

$$p(x \mid y) \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} e^{-\frac{1}{2}(x-\mu)^T \Sigma^{-1} (x-\mu)}$$

$$\Rightarrow p(x \mid y = 0) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} e^{-\frac{1}{2}(x-\mu_0)^T \Sigma^{-1} (x-\mu_0)} \quad x \in \mathbb{R}^{d \times 1}$$

$$P(x) = e^{-\frac{1}{2}(x-\mu_0)^T \Sigma^{-1}(x-\mu_0)} \times P(y=0) + e^{-\frac{1}{2}(x-\mu_1)^T \Sigma^{-1}(x-\mu_1)} \times P(y=1)$$

$$\Rightarrow e^{-\frac{1}{2}(x-\mu_0)^T \Sigma^{-1}(x-\mu_0)} (1-\phi) = e^{-\frac{1}{2}(x-\mu_1)^T \Sigma^{-1}(x-\mu_1)} \phi$$

$$e^{\frac{1}{2}(x-\mu_0)^T \Sigma^{-1}(x-\mu_0) + \frac{1}{2}(x-\mu_1)^T \Sigma^{-1}(x-\mu_1)} = \frac{\phi}{1-\phi}$$

$$\frac{1}{2}(x-\mu_1)^T \Sigma^{-1}(x-\mu_1) - \frac{1}{2}(x-\mu_0)^T \Sigma^{-1}(x-\mu_0) = \ln\left(\frac{\phi}{1-\phi}\right)$$

If we just have same Σ and $p(y=0) = p(y=1)$ are same,

we just look at the euclidean distance

$$e^{-\frac{1}{2}(x-\mu_1)^T(x-\mu_1)}$$ squared euclidean dist from μ_1

$$e^{-\frac{1}{2}(x-\mu_2)^T(x-\mu_2)}$$ from μ_2

$\Sigma = I$

Σ = same / const

Say 3 classes.

$p(x \mid y=0) = p(x; \mathcal{N}(\mu_0, \Sigma))$

$p(x \mid y=1) = \mathcal{N}(\mu_1, \Sigma)$

$p(x = 1 \mid y=2) = \mathcal{N}(\mu_2, \Sigma)$

$y \sim \cdot p_1, p_2, 1-p_1-p_2$

Derive classification rule for GDA.

MLE for GDA

$p(x \mid y=0) = \mathcal{N}(\mu_0, \Sigma)$

$p(x \mid y=1) = \mathcal{N}(\mu, \Sigma)$

$y \sim$ Bernoulli.

$p(y|x)$ x of $x_1, \ldots, x_n$ are iid ~~distributed~~ samples

$x_1 \ldots x_N$ $y_1 \ldots y_N$

$$\ell(\phi, \mu_0, \mu_1, \Sigma) = \prod_{i=1}^{N} p(x_i \mid y_i) p(y_i)$$

$$\ell(\quad) = \prod_{i=1}^{n} p(y_i, x_i; \phi, \mu_0, \mu_1, \Sigma)$$ joint conditional prob.

$$\ell(\phi, \mu_0, \mu, \Sigma) = \prod_{i=1}^{n} p(x_i \mid y_i) p(y_i)$$

Class conditioned probability distribution

model - distribution for $y = 1$ (say) or $y = 2$.

Stanford ML Notes.

Summary

Binary classifier

$y = f(x; \omega) : R^d \to 4$

$x \in R^d$

$y \in \{0, 1\}$

Probabilistic classifier

$f(\vec{x}; \omega) : R^d \to P(y|x)$

for binary classification $P(y|x)$ is usually modelled as a Bernoulli distribution

$P(y=1|x) = p$

$P(y=0|x) = 1-p$

For M-ary classification

$P(y|x)$ is a probability mass function.

$P(y|x) = [p_1 \; p_2 \; -- \; p_M]$

$p_i \geq 0 \quad \sum_{i=1}^{M} p_i = 1.$

Two categories of classifiers

1) Discriminative classifiers.

Directly predict $P(y|x)$ or just directly y given x

Eg logistic regression

$P(y|x;\omega) = f(x;\omega) = \frac{1}{1+e^{-\omega^T x}}$

2) Generative classifier

Model the joint distribution $P(x,y)$ of data and infer $P(y|x)$ using Bayes' Rule

$P(y|x) = \frac{P(x|y)\,P(y)}{P(x)}$

$P(x|y)$: class conditional data distribution

$P(y)$: The a priori class probability

$(P(x,y) = P(x|y)\,P(y))$

Need for generative classifiers

- Digit recognition, classify a binary image of a handwritten image as either 5 or 6.

1) Binary classification problem $y \in \{5, 6\}$ & our input is, say image of 30 x 30 binary pixels

$\vec{x} = \{x_1, x_2 - - - x_n\} \quad n = 900$

$x_i \in \{0, 1\} \quad i = 1 - - n.$

Directly estimating $P(y|\vec{x})$ may require answering awkward questions like:

given $\vec{x} = 0$ i.e. $x_1 - - = x_{900} = 0$.

what is

$P(y|x=0)$

i.e. $P(y=5|x=0) = p$

$P(y=6|x=0) = 1-p$ } What is p?

How to estim

This is hard as we won't have enough training samples with exactly $x = 0$ so better to check the likelihood under each class $p(x|y)$ and then use it to estimate posterior as

c) $P(y|x) = \frac{P(x|y)P(y)}{P(x)}$ Generative classifier

$$P(x) = \sum_{j=4} P(x|y=j)P(y=j)$$

Probability of the input x, expressed as the marginal distribution over class variable y.

$P(x|y)$ is easier to estimate, esp Naive Bayes where we assume conditional independence of all input features of the input X given the class.

$$P(x_1, x_2, \dots x_N | y) = \prod_{i=1}^{N} P(x_i|y)$$

+) Often estimating $P(x_i|y)$ is much easier to estimate.

We assumed that $x_i \in \{0,1\}$, now we predict for GDA

$D = \{x_1, x_2, \dots x_n\}$ $x_i \in R^d$

$P(x, y | \mu_1, \mu_0, \Sigma, \Sigma_0, \phi)$.

joint distribution

we don't know y.

So we need to find $p(x|y=i)$ for generative models.

We define z as another variable

$z \in \{0,1\}$.

$P(x, z; \mu_0, \mu_1, \Sigma_0, \Sigma_1, \Phi)$

$z \in \{0,1\}$

$= P(x|z; \mu_0, \mu_1, \Sigma_0, \Sigma_1, \phi] P(z; \phi)$.

very imp to model underlying data distribution.

we are doing this by mixture of gaussians.

$\text{Max } l(\omega) = \prod_{i=1}^{N} P(x_i; \omega)$

$\Rightarrow$ we need to know distribution of x_i.

K means Objective fn is within cluster variance (sum over all clusters)

$$\sum_{i=1}^{N}\sum_{k=1}^{K} r_{ik}\,\|x_i - \mu_k\|_2^2$$

$$\mathbb{E}\left[(x-\mu)^2\right]$$

- Depends on initialisation.
  Do some random initialisations

~~$p(x_i;\omega) = \sum_{i=1}^{K} p_i$~~

$$l(\omega) = \prod_{i=1}^{N} p(x_i;\omega)$$

$$= \prod_{i=1}^{N} p(x_i \mid z_i;\omega)\, p(z_i;\omega)$$

$$= \prod_{i=1}^{N}\sum_{k=1}^{K} p(x_i \mid z_i=k;\omega)\, p(z_i=k)$$

$$\log l(\omega) = \sum_{i=1}^{N} \log \sum_{k=1}^{K} p(x_i \mid z=k)\, p(z=k)$$

$p(x_i \mid z_i=k) \Rightarrow$ can be anything

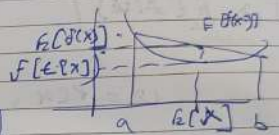
When it's a gaussian, we get a Mixture of Gaussians or Gaussian Mixture Models

---

Jensen

$$p(x) = \sum_{k=1}^{K} \underbrace{\pi_k}_{p(z=k)}\, \underbrace{\mathcal{N}(\mu_k, \Sigma_k)}_{p(x \mid z=k)}$$

We have to extract the meaningful rep. from your data for x. x could be very noisy

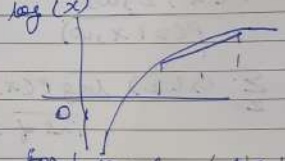


Convex
$\forall \alpha \in [0,1]$
$\forall a, b \in \text{dom}(f)$

$\alpha f(a) + (1-\alpha) f(b)$

Jensen's inequality $\geq f(\alpha a + (1-\alpha) b)$

$\mathbb{E}[f(x)] \geq f[\mathbb{E}(x)]$



⇒ Now Jensen's ineq. $\mathbb{E}[f(x)] \leq f[\mathbb{E}[x]]$

for 1 sample ($N=1$)

$$L(\omega) = p(x;\omega)$$

$$\log L(\omega) = \log \sum_{k=1}^{K}\sum_{z=1} p(x, z;\omega)$$

$$= \log \sum_{z} q(z)\, \frac{p(x, z;\omega)}{q(z)}$$

$$= 1$$

$q(z) \geq 0$

$\sum_z q(z) = 1$

$$= \log \mathbb{E}_{z\sim q(z)}\left[\frac{p(x, z;\omega)}{q(z)}\right]$$

$L(\omega) = \log \sum_z Q(z) \frac{P(x, z; \omega)}{Q(z)}$

$\geq \sum_z Q(z) \log \frac{P(x, z; \omega)}{Q(z)}$ — by Jensen's inequality

$\log[\mathbb{E}_z[u]] \geq \mathbb{E}_z[\log(u)]$

$u = P(x, z; \omega)$

$Q(z) = P(z \mid x; \omega)$

$u = \frac{P(x, z; \omega)}{P(z \mid x; \omega)}$

$L(\omega) \geq \sum_z Q(z) \log P(x; \omega)$

ind of z

$\geq \log P(x; \omega) \sum_z Q(z)$

$L(\omega) \geq \log p(x; \omega)$ — (2)

⇒ we started with $\log P(x; \omega) = L(\omega)$

⇒ 2 becomes $L(\omega) = \log P(x; \omega)$

Recap

$\mathbb{E}[\log(u)] \leq \log(\mathbb{E}[u])$

$\sum_z Q(z) \log \frac{P(x, z; \omega)}{Q(z)} \leq \log \sum_z Q(z) \frac{P(x, z; \omega)}{Q(z)}$

$\Rightarrow \leq \log P(x; \omega)$

$\Rightarrow L(\omega) \geq \sum_z Q(z) \log \frac{P(x, z; \omega)}{Q(z)}$

when $Q(z) = P(z \mid x; \omega)$

$\Rightarrow L(\omega) \geq \sum_z P(z \mid x; \omega) \log \frac{P(x, z; \omega)}{P(z \mid x; \omega)}$

$\Rightarrow L(\omega) \geq \sum_z P(z \mid x; \omega) \log \frac{P(x; \omega) P(z \mid x; \omega)}{P(z \mid x; \omega)}$

$= \sum_z P(z \mid x; \omega) \log P(x; \omega)$

$= \log P(x; \omega) \sum_z P(z \mid x; \omega)$

$= \log P(x; \omega)$

$\Rightarrow L(\omega) \geq \log P(x; \omega)$

Hence as we started with $L(\omega) = \log P(x; \omega)$ the Evidence Lower Bound (ELBO) becomes equal to $\log P(x; \omega)$ i.e. $L(\omega)$

1) $Q_i(z^{(i)}) = P(z \mid x^{(i)}, \omega)$

$\log P(x^{(i)}; \omega) \geq ELBO(x^{(i)}, Q_i, \omega)$

$\Rightarrow \sum_{z^{(i)} \sim Q_i(z^{(i)})} Q_i(z^{(i)}) \log \frac{P(x_i, z_i; \omega)}{Q_i(z_i)}$

We want:

$\max_{\omega} \log P(x^{(i)}; \omega)$ (One sample i)

$\log P(x_i; \omega) = \log \sum_{k=1}^{K} \pi_k N(x_i, \mu_k, \Sigma_k)$

$ELBO(x_i, Q_i, \omega) = \sum_{z^i \sim Q_i(z_i)} Q_i(z^i) \log \frac{P(x_i \mid z_i; \omega) P(z_i; \omega)}{Q_i(z_i)}$

We can differentiate this exp, given a z_i.

2) At time t:

1) Set $Q_i(z_i) = P(z_i \mid x_i; \omega^t)$ from bayes rule

2) $\omega_{t+1} =$ Maximise $ELBO(x_i, Q_i, \omega)$

(Lower bound on likelihood) is calculated for $t+1$, using ω_t to optimise

Even with ω_t this is still a lower bound.

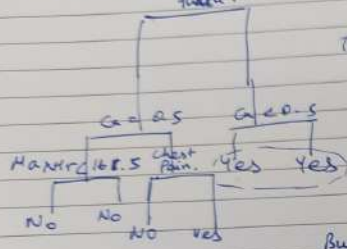
So the $\log P(x_i; \omega)$ is hard to optimise directly, hence we instead derive a lower bound using Jensen's inequality and maximise that instead

The EM algo does converge but not to a global optimum everytime

Initialisation is also a large factor here

Intro to Statistical Learning Chapter 8.

Can have the same attribute appear multiple times in a decision tree.



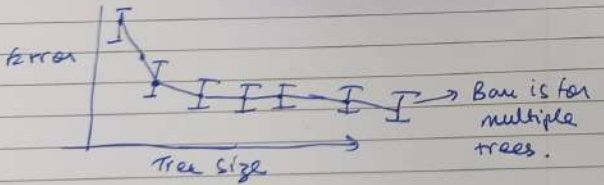
This is a pruned tree.

So this is redundant for a Yes/No But it could be a probability score for the class.

Previously left branch did have some NO as well.

2) left branch can have say 20% misclass and right could have 0% misclassified.

2) Cross validation to decide the depth of the tree



- So after pruning we still retain the idea of misclassification of train error.

If we just decide as soon as that - a right branch:

Entropy may rise
Misclassification error is higher
but splitting it can reduce it.
(One branch has 0 entropy) or lower.

Decision Variable selection

Drop in Entropy:

$$H(Y) = -\sum_{ic=1}^{ic} P(Y=y_{ic}) \log P(Y=y_{ic})$$

Drop in Gini

$$G(Y) = \sum_{ic=1}^{ic} P(Y=y_{ic})(1-P(Y=y_{ic}))$$

Trees Adv and Disadv

- Very easy to explain
- Easy to visualise graphically and can be interpreted even by a non expert.
- Can handle both discrete and continuous feature without need to create dummy variables
  One hot encoding Ex Weather: cloudy sunny
- Generally don't achieve same level of accuracy as some other approaches

2) Greedy Approach, Also why they tend to overfit: Get diff trees for diff subsamples.

can change this via Ensembles.

Bagging : Bootstrapped Aggregation
Boosting

Bootstrapping

- Method to reduce variance
  similar to sample mean & getting tighter (less variance) as no of samples grows.

$x_1, x_2 \dots x_N$ i.i.d.

$\bar{x} = \frac{1}{N} \sum_{i=1}^{N} x_i$

$E[x] = \mu$

$Var(x) = \sigma^2$.

$\Rightarrow E[\bar{x}] = \frac{1}{N} \sum_{i=1}^{N} \mu = \mu$.

$Var(\bar{x}) = \frac{1}{N^2} \sum_{i=1}^{N} Var(x_i)$

$= \frac{1}{N^2} N \times \sigma^2 = \frac{\sigma^2}{N}$.

$= \square$

So you would have N independent samples of the data and then train N decision trees, and predict as the average of all predictions.

But you would just train on the entire data then.

Often don't have this much data.

⇒ Bootstrap
  sample with replacement.

2) Get B datasets (could have repeated data points)
(surrogates for the ind datasets).
  Estimate statistic for all B and average it.

2) Reduces variance
  Even if the trees are correlated the drop in variance is good.

Random forest, Boosted Trees can model very complex stuff
  very good classifiers

⇒ The B decision trees are not pruned(?)

2) At inference

$\hat{f}_{bag}(x) = \frac{1}{B} \sum_{b=1}^{B} \hat{f}_b(x)$

⇒ $\hat{f}_1 \quad \hat{f}_2 \quad \dots \quad \hat{f}_b$ trees

2) Say CV for k-fold
  Now we have k B.
  for each fold B bootstrapped models.

Hence CV can be very expensive for ensemble models

Out of Bag Error

Very good replacement for CV.

$X = \{x_i\}_{i=1}^{N}$

X_b = Bootstrapped dataset.

$P(x_b^i = x^j) = \frac{1}{N}$ (random sampling)

$P(x_b^i \neq x^j) = \left(1 - \frac{1}{N}\right)$

Hence x^j not being in the X_b is

$P(x_j \notin X_b) = \left(1 - \frac{1}{N}\right)^N$

(All independent samples in X_b)

$x_b^1 \neq x_j \cdot x_b^2 \neq x_j$

$\cdots x_b^N \neq x_j$.

$\lim_{N \to \infty} \left(1 - \frac{1}{N}\right)^N = \frac{1}{e} \approx 0.368$

Hence for a large enough N, samples doesn't appear in a bootstrapped dataset is 0.36.

=) Out of Bag samples.

So models doesn't see 36% of the original dataset.

=) We have a validation set for the model.

Hence for every tree we have 30% of samples that are not seen.

=) We do prediction on these OOB samples and avg their error to get OOB error (for a model)

Good approx about test error.

=) We calc OOB error for each sample

Approx 30% of B datasets (here) that it is OOB for

Hence

$Error_{OOB}^{(x_i)} = \frac{1}{30} (f_1(x_i) + \cdots + f_{\frac{30}{100}N}(x_i))$

=) for all $x_i = x_N$

=) Bagging reduces variance and overfitting But now interpretability is lost.

But can still track Importance of variables for Bagged DTs

=) Every time a split is done using a feature track reduction in error. Average it.

Random forests

- Same as bagging, but decorrelated trees (to inc the red in var)

At each split
Restrict to decision var to $\sqrt{p}$ of the variables.
(whichever has lowest entropy / All predictors) Gini variables
Allow different decision trees to explore other splits rather than those that reduce the error the most.

Boosting Decision trees

~~Bagging~~ ~~Resampling to~~

Bagging : Trees are trained ~~ind~~ and at once.

Boosting : Sequentially
Now previous trees are used to ~~train~~ new trees.

Benefits most from weak learners: decision stumps
The trees are simple and they sequentially train on ~~predict~~ the residuals.

final is $\hat{f}(x) = \sum_{b=1}^{B} \lambda \hat{f}_b(x)$

Can become very powerful.

Ada Boost For classification

Focus on what prev model misclassified.

General Additive models.

$\hat{f}(x) = \hat{f}(x) + \lambda \hat{f}_b(x)$

↳ shrinking factor
Fixed prior (usually)

$r_i = r_i - \lambda \hat{f}_b(x_i)$

How much we updated model by

How much is actually left to explain.

Ada Boost

Initialise $w_i = \frac{1}{N}$ $(i = 1, 2, \ldots N)$

⇒ Equally hard. initially for all samples.

for $m = 1$ to M:
(a) Fit a classifier $G_m(x)$ to train data using w_i.
(loss is calculating w_i)

(b) Compute Error$_m$ $= \dfrac{\sum_{i=1}^{N} w_i \, I(y_i \neq G_m(x_i))}{\sum_{i=1}^{N} w_i}$

(c) Compute $\alpha_m = \log\left(\frac{(1-err_m)}{err_m}\right)$

$\Rightarrow w_i \leftarrow w_i e^{\alpha_m \mathbb{I}\{y_i \neq c_m(x_i)\}}$

$i = 1, 2, \ldots N.$

Final decision

$$c(x) = \text{sign}\left[\sum_{m=1}^{M} \alpha_m c_m(x)\right]$$

Abstraction for boosting methods

$f_0(x) = 0.$

for $m = 1$ to M:

(a)

$$(\beta_m, \gamma_m) = \arg\min_{\beta, \gamma} \sum_{i=1}^{N} L(y_i, f_{m-1}(x_i) + \beta\, b(x_i; \gamma))$$

The model we want at step m

(b) $f_m(x) = f_{m-1}(x) + \beta\, b(x; \gamma_m)$.

Predicted value for the regions

Boosting Trees

$$T(x; \Theta) = \sum_{j=1}^{J} \gamma_j I(x \in R_j)$$

$$\Theta = \{R_j, \gamma_j\}_1^J$$

Partition of feature space into J regions (R_j)

γ_j : Pred val of region.

$$\hat{\Theta} = \arg\min_{\Theta} \sum_{j=1}^{J} \sum_{x_i \in R_j} L(y_i, \gamma_j).$$

combinatorial → needs

Step 1 : find regions R_j

2) Find γ_j.

2) Bagged $= \sum_{m=1}^{M} T(x; \Theta_m)$

Gradient Tree boosting.

$$f_0(x) = \arg\min_{\gamma} \sum_{i=1}^{N} L(y_i, \gamma)$$

$m = 1$ to M

(a) for $i = 1, 2, \ldots N$

$$r_{im} = -\left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)}\right]_{f = f_{m-1}}$$

We want to reduce the loss function.

wrt the prediction.

At f_{m-1}

c) Gradien

decision tree

(b) h_t to targets R_{jm} giving terminal regions R_{jm}

$j = 1, 2, \ldots J_m$.

(c) for $j = 1, 2, \ldots J_m$

$$\gamma_{jm} = \underset{\gamma}{\operatorname{argmin}} \sum_{x_i \in R_{jm}} L(y_i, (f_{m-1}(x_i) + \gamma))$$

(d)

$$f_m(x) = f_{m-1}(x) + \sum_{j=1}^{J_m} \gamma_{jm} I(x \in R_{jm})$$

⇒ Again weak learners benefit boosting

$$x = \{x^{(1)}, x^{(2)}, \ldots x^{(N)}\}$$

$$y = \{y^{(1)}, y^{(2)}, \ldots y^{(N)}\}$$

$x_i \in R^d$

$y_i \in \{-1, 1\}$

$$P(y_i = 1 \mid x_i) = \sigma(w^T x + b)$$

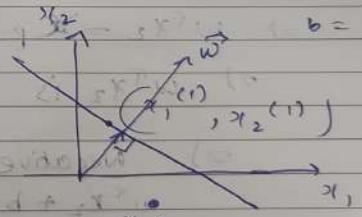
$x_2 = m x_1 + c \Rightarrow m x_1 - x_2 + c = 0$

$$\begin{bmatrix} m & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + c = 0$$

$w_1 = m$, $w_2 = -1$, $b = c$ ⇒ line is $w^T x + b = 0$.

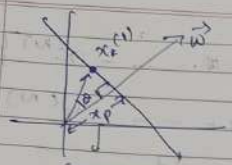
$$\vec{w} = \begin{bmatrix} m \\ -1 \end{bmatrix} = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}$$

$b = c$.



$w_1 x_1 + w_2 x_2 + b = 0$.

$\Rightarrow w^T x + b = 0$



$w^T x_p^{(i)} + b = 0$ (Lies on the line)

Component of x along w.

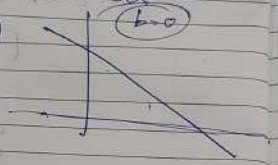
$x_p = \|w\| \|x^{(i)}\| \cos\theta \geq 0$

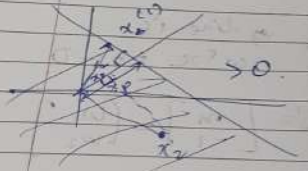
$\Rightarrow b = -x_p$.

Assume $\|w\| = 1$

$\frac{1}{\sqrt{2}} x_1 + \frac{1}{\sqrt{2}} x_2 + b = f(x; b)$

$x_1 + x_2 + 1 = 0$





> 0

< 0

a) b was $= -x_p$

b) $w^T x_2 - x_p$

c) $w^T x_2$ is $< x_p$

d) Negative valued $w^T x_2 + b < 0$.



SVM: Stanford Lecture Note

$\sigma(w^T x + b) = \frac{1}{1 + e^{-(w^T x + b)}}$.

a) As we go away (right) then $w^T x + b$ is very large and +ve

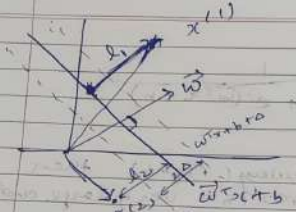
b) $\frac{1}{1 + e^{-(w^T x + b)}} \to 1$

else (left) $w^T x + b$ is very negative.

$\frac{1}{1 + e^{-(w^T x + b)}} \to 0$.

- Usual models go for the whole data likelihood and Loss.

- SVMs do constrained optimisation. Not affected by outliers.



$x = x^{(1)}$

$l^{(1)} = \frac{w^T x^{(1)} + b}{\|w\|}$

$l^{(2)} = -\frac{w^T x^{(2)} + b}{\|w\|}$

signed distance.

$l^{(i)} = y^{(i)} \cdot \frac{w^T x^{(i)} + b}{\|w\|}$

$(w^T x + b - \Delta) \cdot \|w\|$

$(w^T x + b + \Delta) \|w\|$

we are in binary classification.

We want to maximize the margin (2Δ) that correctly classifies the points.

max 2Δ ~~s.t.~~ ~~$y^{(i)}(w^T x^{(i)} + b) \geq \Delta$~~

s.t. $y^{(i)}(w^T x^{(i)} + b) \geq \Delta \quad \forall i$ (neglect $\|w\| \geq 0$)

$\|w\| = 1$)

So the $l^{(i)}$ should be greater than 0 i.e.

2) $\|w\|_2 = 1$

This is only if data is linearly separable.

$i \in \{1, \dots N\}$

↓

The magnitude of $y^{(i)} w^T x^{(i)} + b$ should be greater than Δ

Don't want to be so close to the decision boundary.

(Not within $\pm\Delta$.)

⇒ max 2Δ s.t. $y^{(i)}(w^T x^{(i)} + b) \geq \Delta$

w, b $\|w\|_2^2 = 1$.

$\forall i \in \{1, 2, \dots N\}$

If $\|w\|_2^2$ is not there just change w to make $(w^T x^{(i)} + b) y^{(i)} \geq \Delta$ to ~~change~~ hold.

- This is a naive representation of data.
  - Hard to solve
  - Non-convex

Now $\|w\|_2^2 = 1$

Means we have to search in a unit circle ($d = 2$)



So if you do gradient ascent / descent here, we may step out of the unit circle.

Convex Optimisation

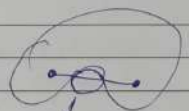
- Can efficiently find global solution for complex functions. Maxima / Minima
  - ⇒ The local minimal maxima is the global maxima / minima. Also unique (Not necessarily)
  - ⇒ Initialisation is hence not a concern
    - KMeans, GMM need multiple initialisation
    - Not convex optimisation for KMeans, GMM.

Convex Sets

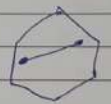
A set C is convex if for any $x, y \in C$ and $\theta \in \mathbb{R}$ with $\theta \in [0,1]$

$\Rightarrow$ $\theta x + (1-\theta) y \in C$.

All points along line from any two elements in C (x to y) lies in C.



doesn't lie in C



Convex.

linear subspace : Has to pass through origin.

Affine subspace : No such restriction ($Ax = b$ also convex) $w^T x + b$: Affine subspace.

Intersection of convex sets / subspaces is convex

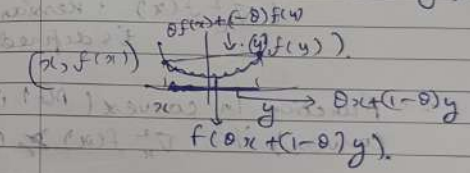
Positive semidefinite : Non neg eigenvalues $\Rightarrow$ Ex $A^T A$.

Convex functions

A function $f : \mathbb{R}^n \to \mathbb{R}$ if domain ($\mathrm{Dom}(f)$) is convex.

and for all $x, y \in \mathrm{Dom}(f)$ and $\theta \in \mathbb{R}$, $\theta \in [0,1]$

$$f(\theta x + (1-\theta) y) \leq \theta f(x) + (1-\theta) f(y)$$



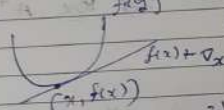
Can do the same for higher dimensional domains and functions.

First order condition for convexity

Suppose $f: R^n \to R$ differentiable ($\nabla_x f(x)$ exists)

$\Rightarrow$ f is convex iff $D(f)$ is convex and $\forall\, x, y \in D(f)$

$$f(y) \geq f(x) + \nabla_x f(x)^T (y - x)$$



first order approximation

$\Rightarrow$ Tangent line is a global underestimator of the function f.

Second Order Condition

let $f: R^n \to R$ & is twice diff ($\nabla_x^2 f(x)$: Hessian i.e. it's defined)

$\Rightarrow$ Function in convex ($D(f)$ is convex) iff $\nabla_x^2 f(x) \succeq 0$

Positive semi definite

Jensen's Inequality

$$f(\mathbb{E}[x]) \leq \mathbb{E}[f(x)]$$

Convex combination is basically expectation (All Sum to 1 $\theta \in [0,1]$)

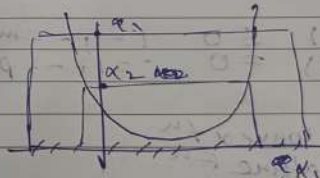
Sublevel set

Convex func give a partical convex set :

α - sublevel set

Curve $f: R^n \to R$: convex fn and a real number $\alpha \in R$, α-Sublevel set :

$$\alpha\text{-S-L} = \{x \in D(f) : f(x) \leq \alpha\}$$



α_1-sublevel set.

2) Si let $x, y \in \alpha$-sublevel set

$\Rightarrow$ $x, y \in D(f)$ s.t. $f(x) \leq \alpha$ $f(y) \leq \alpha$.

Then

$$f(\theta x + (1-\theta)y) \leq \theta f(x) + (1-\theta) f(y)$$
$$\leq \theta \alpha + (1-\theta)\alpha$$
$$= \alpha.$$

$\Rightarrow$ $\theta x + (1-\theta)y \in \alpha$ - sublevel set

$\Rightarrow$ α - sublevel is a convex fn.

Convex Optimisation Problem

$$\min f(x)$$
$$\text{s.t.}$$
$$x \in C$$
↳ convex set.
f : convex fn.

$$\min f(x)$$
$$\text{s.t.}$$
$$g_i(x) \leq 0 \quad i = 1, \dots, m$$
$$h_i(x) = 0 \quad i = 1, \dots, p$$

$g_i(x)$: convex fn
$h_i(x)$: affine fn

$\Rightarrow$ Without constraints x is the entire domain of $f(x)$

But with constraints we limit the feasible solutions : x that satisfies all constraints

Optimal would be one that is feasible and minimizes $f(x)$.

The $g_i \leq 0$ 0 - sublevel set may to be less than the intersection of the constraint subspaces (affine are also comp'ex) will not longer be convex
$\Rightarrow$ Not guaranteed to find global optima.

Only affine func can be equality.
i.e $h_i = 0$ ⇒ $g_i \leq 0$
$h_i \geq 0$
$\Rightarrow$ h_i is concave & ~~affine~~ convex as affine.

Global Optimality
x is a:
local optimum
with in the neighbourhood R.
s.t. $\|x - z\|_2^2 \leq R$.
$f(x) \leq f(z)$
(for a minim problem)
global $\forall z$
$f(x) \leq f(z)$

Convex opt problem: All locally optimum points are globally optimum.

Proof by contradiction

Say a feasible x is locally optimum, but there is a feasible y that has a lower objective value.

$\therefore f(x) > f(y)$

By def of local optima, $\nexists$ z (feasible) s.t. - $\|x - z\|_2 \leq R$ and $f(z) < f(x)$

Say $z = \theta y + (1-\theta)x$ $\quad \theta = \frac{R}{2\|x-y\|_2}$, < 1

$\|x - z\|_2^2$

$= \left\| x - \left(\frac{R}{2\|x-y\|_2} y + \left(1 - \frac{R}{2\|x-y\|_2}\right) x \right) \right\|_2$ (y is beyond the neighbourhood)

$= \left\| \frac{R}{2\|x-y\|_2} (x-y) \right\|_2$

$= \frac{R}{2} \leq R \Rightarrow z$ is in neighbourhood.

By convexity of f: (x, y, z are feasible)

$f(z) = f(\theta y + (1-\theta)x)$

$\leq \theta f(y) + (1-\theta) f(x)$

$< \theta f(x) + (1-\theta) f(x)$

$< f(x)$

$\Rightarrow$ z is feasible as x and y are feasible and $z = \theta y + (1-\theta)x$.

$\Rightarrow$ z is feasible $\|x - z\|_2 \leq R$

but $f(z) < f(x)$

Hence contradiction to our assumption that x is locally optimum.

3. (5 points) Linear Regression: Derivation

Gra